

木幅学習システムの構築

高効率なグラフ解析に向けた木幅概念の学習システム構築

Development of a Learning System for Treewidth

Constructing a System for Acquiring the Concept of Treewidth for Highly Efficient Graph Analysis

あらまし 木幅は、グラフ理論やアルゴリズム設計において重要な概念である。しかし、木幅の抽象的な概念や日本語で学べる教材が少ないなどという問題がある。したがって、今回は木幅や木分解の定義理解を促進する Web アプリケーションによる学習システムを構築した。

キーワード 木幅, 木分解, Next.js, ReactFlow

1. はじめに

グラフ理論 [1] は、ディペンダブルコンピューティングの分野では、信頼性解析に、ソフトウェアの分野では、コンパイラのレジスタ割り当て、スケジューリングなど多くの分野の基礎的な理論として重用されてきた。

一般に解くのが難しい問題でも、グラフが木であれば容易に解けることがわかっている問題も多い。木幅は、グラフ理論やアルゴリズム設計において一つの重要な概念である。木幅とは、そのグラフがどれだけ木に近いかを表す指標である。グラフを木分解とは、部分集合（バッグと呼ぶ）に分けて木で接続する方法で、その部分集合の最大要素数 - 1 を木幅と定義している [2]。これが小さいと、木分解のバッグに含まれる超点数が少ないので、効率的に解けることが知られている [2]。例えば、最大独立集合問題、グラフ彩色問題、最短経路問題などへの応用が知られている。

また、木幅が小さいと各バッグ内の状態の組み合わせが少ないので動的計画法が使いやす位という特徴もある。これ以外にも多くの問題で木幅の概念を活用すれば、効率よく解けるかもしれない。しかし、木幅の抽象的な概念や木分解の構築の難しさにより認知が低く、日本語で学べる教材や可視的に学ぶことができる機会

が少ない。そこで、木幅や木分解の定義の理解を促進する学習システムを構築したので、報告する。

2. 木幅 (treewidth) とは

本節では、グラフ理論の基本的な用語について概説する。詳細は文献 [2] を参照されたい。

2.1 木と閉路

閉路とは、始点と終点が一致し、それ以外の頂点を重複せずに通る道である。グラフ理論において、閉路の存在はグラフ構造の複雑さを特徴づける重要な要素の一つであり、多くの計算問題において困難性の原因となる。

一方、木とは、閉路を含まず、かつ連結である無向グラフである。木は極めて単純な構造を持つグラフであり、以下の性質が知られている [1]。

- 木は閉路を含まない連結グラフである。
- 任意の二頂点間にただ一つの単純路が存在する。
- 任意の辺を一つ削除すると、グラフは非連結になる。

図??に木と閉路のあるグラフを示す。

特に、任意の二頂点間に単純路が一意に定まるといふ性質は、木構造における情報の伝播や分割を容易にしている。さらに、木において葉（次数 1 の頂点）以外の任意の頂点を一つ削除すると、グラフは複数の連結成分に分割される。この性質により、木は自然な再帰構造を持ち、分割統治法や動的計画法を適用しやす

†

DOI:10.14923/transj.???????????

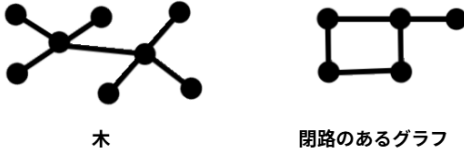


図1 木と閉路のあるグラフ

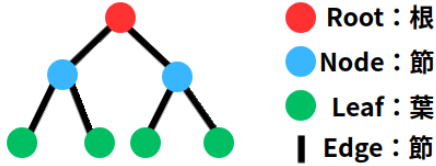


図2 木構造の用語

い対象となる。このような構造的単純性のため、最大独立点問題など多くの計算困難な問題であっても、入力グラフが木である場合には多項式時間、あるいは線形時間で解くことが可能である。一方で、一般のグラフは閉路を多く含み、木のような単純な構造を持たない場合が多い。そのため、任意の一般グラフに対して、直接木と同様のアルゴリズムを適用することは困難である。

そこで、一般グラフを「完全な木」ではなく、「木に近い構造」として捉え、その上で計算を行う枠組みが考えられてきた。この考え方を形式化した概念の一つが木分解 (tree-decomposition) である。

2.2 木分解 (tree-decomposition)

木分解とは、一般のグラフの頂点集合を部分集合の集まりとして分割し、それらの関係を木構造で表現する手法である。これにより、元のグラフの構造的複雑さを、木構造上で扱うことが可能となる。木分解は、一般グラフ上の問題を木構造上の問題として扱うための基盤となる概念であり、を測る指標として木幅が定義される。木分解は以下のように定義される [2]。

[定義 1] V をグラフ G の頂点集合、 E を辺の集合とする。また、 I を木 T の頂点集合、 F を辺の集合とする。グラフ $G = (V, E)$ の木分解 (tree decomposition) とは、木 $T = (I, F)$ と、各 $i \in I$ に対応する頂点集合 $\{X_i \mid i \in I\}$ (bags (バッグ) と呼ばれる) からなる組 $(\{X_i \mid i \in I\}, T)$ であって、次の条件を満たすものをいう。

(TD1) 全ての頂点がどこかのバッグに含まれる。

$$\bigcup_{i \in I} X_i = V$$

(TD2) グラフ G の辺を構成する頂点は必ず同じバッグに含まれる。

$$\forall (u, w) \in E, \exists i \in I \text{ で } u \in X_i \text{ かつ } w \in X_i$$

(TD3) 同一頂点を含むバッグ X は木 T 上で連結である。

$$\forall i, j, k \in I, j \in \text{path}_T(i, k) \Rightarrow X_i \cap X_k \subseteq X_j$$

ここで $\text{path}_T(i, k)$ は、木 T において頂点 i から頂点 k を結ぶ唯一の単純路を意味する。

2.3 木幅 (treewidth)

木分解を用いることで、一般のグラフを木構造として扱うことが可能になるが、その際に重要となるのが「どれだけ小さな部分集合で分解できるか」である。直感的には、木分解における各バッグの大きさが小さいほど、元のグラフは「木に近い構造」を持つと考えられる。

この考えを定量的に表した指標が木幅 (treewidth) である。

[定義 2] 木分解 $(\{X_i \mid i \in I\}, T)$ の幅とは、

$$\max_{i \in I} (|X_i| - 1)$$

で定義される値である。

ここで、各 X_i は木分解におけるバッグを表し、その大きさ $|X_i|$ は同時に扱う必要のある頂点数を意味する。定義において -1 を行うのは、木に対して木幅が 1 となるように調整するためである。

次に、グラフ全体に対する木幅を定義する。

[定義 3] グラフ G の木幅 (treewidth) とは、 G のすべての木分解の幅の最小値である。

木幅はグラフの不変量であり、グラフがどの程度「木に近い構造を持つか」を表す指標である。木幅が小さいグラフほど、木構造上で有効な分割統治法や動的計画法を適用しやすい。

また、木分解は部分木の集合として捉えることもできる。木分解の定義における条件 (TD3) により、ある頂点 $v \in V(G)$ を含むバッグの集合は木 T 上で連結な部分木 T_v を形成する。さらに、条件 (TD2) により、辺 $uv \in E(G)$ に対しては部分木 T_u と T_v が共通のバッグを持つことが保証される。

図 3 の場合、これらの条件を満たす木分解の一例は図 4 のようになる。頂点集合 X_i において (TD1), (TD2)

が満たされていることがわかる。また、図4の点線は、バッグ X が (TD3) を満たしていること示している。したがって、この木分解の幅は2であり、他の木分解の幅も同値となるため、この木幅は2となる。

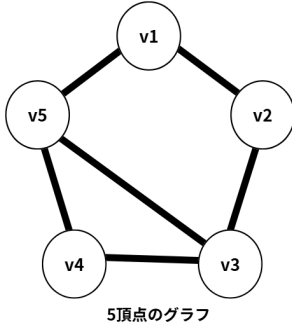


図3 5頂点のグラフの場合

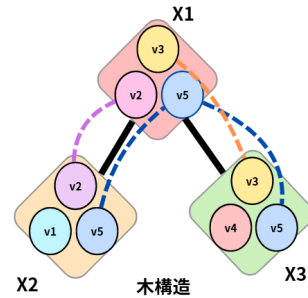


図4 木分解の例

3. 学習支援システムの設計

本研究では、木幅および木分解の定義理解を目的として、問題演習を通じて学習を支援するシステムを設計・実装する。本システムは、木分解に関する定義や性質を段階的に確認できる問題形式を採用することで、学習者の理解を促進することを目的とする。

3.1 設計方針

木幅および木分解の理解には、定義を正確に把握し、それを具体的なグラフ構造に適用する力が求められる。しかし、従来の学習では定義を読んだだけでは理解が定着しにくいという課題がある。

そこで本システムでは、以下の方針に基づいて設計を行った。

- 木分解の定義や性質を、問題形式で確認できるようにする。
- 様々な問題形式を用いて、理解度を即時に確認できるようにする。
- 問題は段階的に難易度を上げ、定義理解から応用へと導く構成とする。
- 画面上でグラフの頂点や辺をクリックでインタラクティブに選択できるようにする。

本システムは、いきなり木分解の構築を求めるのではなく、定義理解を確認する問題演習を通じて、学習者が必要な前提知識を段階的に獲得できる点に特徴がある。また、学習者がインタラクティブにグラフや木構造に触れられるようにするためにはグラフや木構造の頂点や辺の情報を保持しておく必要がある。したがって、問題作成時には画像を挿入するのではなく、ツールを利用してグラフを生成できるようにしている。

3.2 問題形式の構成

本システムでは、主に以下の問題形式を実装して

いる。

3.2.1 ○×問題

○×問題では、木分解や木幅に関する命題を提示し、学習者がその正誤を選択する。これにより、定義の誤解や曖昧な理解を顕在化させることができる。

例として、「あるグラフの木幅が1であるとき、そのグラフは必ず木である」といった命題を扱う。

3.2.2 入力問題

入力問題では、学習者が数値や簡単な記号を入力することで解答を行う。例えば、提示されたグラフに対して、その木幅や特定の木分解における幅を求めさせる問題を用意している。この形式により、単なる選択ではなく、定義に基づいた計算や判断を要求することができる。

3.2.3 選択問題

選択問題では、複数の選択肢の中から正しいものを選ばせることで、木分解や木幅に関する理解を確認する。この形式は、単純な正誤判定よりも多様な誤答パターンを提示できるため、学習者が自身の誤解の内容を把握しやすいという利点がある。

例えば、木分解の条件 (TD1–TD3) に関する説明のうち、正しいものを選択させる問題や、提示された木分解が正しいかどうかを選択肢から判断させる問題を扱っている。

3.3 使用技術

本 Web システムの実装には、表 1 に示す技術を用いた。以下に、各技術の選定理由を述べる。

表 1 本 Web システムで使用した技術

区分	使用技術
実行環境	Node.js
プログラミング言語	TypeScript
フレームワーク	Next.js
UI ライブラリ	React
データベース	SQLite
ORM	Prisma
グラフ描画ライブラリ	ReactFlow

Node.js [3] は、JavaScript という言語を用いてサーバー側の処理を実行できる環境である。非同期処理を効率よく行える点から採用した。また、フロントエンド (ブラウザで動く部分) と同じ言語でサーバー処理を書けるため、開発が容易である。Next.js [4] と組み合わせることで、ページ遷移やデータ管理の実装が簡単になり、Web アプリケーション全体の構築効率が向上する。

TypeScript [5] は、JavaScript に型付けの機能を追加

した言語である。型付けによってプログラムのミスを防ぎやすくなり、複雑なデータ構造を扱う本システムにおいて安全性と保守性を向上させる。

Next.js は React [6] を基盤とした Web フレームワークであり、Web ページの構築や状態管理を容易にする。本研究では、学習者がグラフ構造を操作する画面を構築する際に便利であるため採用した。

React はコンポーネント指向のユーザインタフェース (UI) ライブラリであり、画面の状態変化に応じて動的に表示を更新できる。これにより、学習者が操作した結果をリアルタイムに画面へ反映できる。

SQLite [7] は軽量な組み込み型データベースであり、追加のデータベースサーバを必要としない。本研究では小規模なデータ管理を想定しており、SQLite で十分な性能を得られる。

Prisma [8] は TypeScript と親和性の高い ORM (データベース操作をコードで簡単に行える仕組み) である。これにより、データベースのスキーマとアプリケーションコード間の不整合を防ぎつつ、安全にデータ操作を行える。

ReactFlow [9] は、ノードとエッジからなるグラフ構造を直感的に描画・操作できるライブラリである。本研究では、木分解や木幅の学習支援を目的として学習者がグラフを動的に編集・確認できる点で有用である。

3.4 実装状況

本システムでは、現時点で以下の機能を実装している。

- ・ 問題提示 (○×・入力・選択問題)。
- ・ 解答・正誤判定。
- ・ 問題ごとの解説表示。
- ・ インタラクティブなグラフを含めた問題追加。

4. 今後の課題

- ・ 問題生成の自動化
- ・ 学習者の解答履歴を用いた理解度分析
- ・ 複雑なグラフ構造や木分解の操作の拡張

5. まとめ

謝辞

文 献

- [1] Robin J Wilson. *Introduction to graph theory*. Pearson Education India, 1979.
- [2] Neil Robertson and Paul D. Seymour. Graph minors. ii. algorithmic aspects of tree-width. *Journal of algorithms*, 7(3):309–322,

1986.

[3] OpenJS Foundation and Node.js contributors. Node.js® とは. <https://nodejs.org/ja/about>, 2026. Accessed: 2026-2-11.

[4] Inc. Vercel. Next.js docs. <https://nextjs.org/docs>, 2025. Accessed: 2026-2-11.

[5] Microsoft. Typescript for the new programmer. <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html>, 2026. Accessed: 2026-2-11.

[6] Inc Meta Platforms. React reference overview. <https://ja.react.dev/reference/react>, 2026. Accessed: 2026-2-11.

[7] SQLite Consortium. About sqlite. <https://www.sqlite.org/about.html>, 2025. Accessed: 2026-2-11.

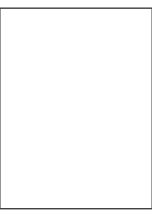
[8] Inc. Prisma Data. What is prisma orm? <https://www.prisma.io/docs/orm/overview/introduction/what-is-prisma>, 2025. Accessed: 2026-2-11.

[9] xyflow team. Overview: Terms and definitions. <https://reactflow.dev/learn/concepts/terms-and-definitions>, 2026. Accessed: 2026-2-11.

付 録

1.

(xxxx 年 xx 月 xx 日受付)



Abstract

Key words